HyperTensor (Part III): Composing Compression with Geodesic Speculative Decoding

William Ken Ohara Stewart
NagusameCS Independent Research
nagusamecs@gmail.com https://github.com/NagusameCS/HyperTensor

April 2026 (v0.1)

Abstract

What happens when you compose calibration-free Geodesic Projection (GP) compression with exactness-preserving speculative decoding [1, 2]? This paper reports the first end-to-end measurement of the combined runtime (`geodessical`) on SmolLM2-135M-Instruct (Q8_0, ChatML). We derive a closed-form throughput model for the composition, separating the win from verifier amortisation, the win from drafter compression, and the loss from compression-induced acceptance-rate degradation, and identify the deployment shapes where the composition wins and where it is a wash on consumer hardware. Along the way we document a previously undocumented failure mode of greedy speculative decoding against instruct-tuned backbones (instruct-greedy-EOS) and the two-line fix (`llm_topk_excluding` plus a four-token min-response guard) shipped in the runtime. Headline measurement: α=38.5% acceptance, 76.5 tok/s end-to-end at γ=4, threshold 0.45, against a baseline of $\sim$ 50 tok/s greedy-only on the same model and binary, a 1.53× measured speedup, within the closed-form prediction. On a second model, Meta-Llama-3.1-8B-Instruct Q4_K_M, we measure α=46.9% at γ=2 on an eight-prompt sweep, with zero per-prompt acceptance variance under deterministic decoding—the verifier accepts geodesic drafts at a rate set by the model's residual-stream geometry rather than by prompt content. We also describe three additional drafter modes shipped in the runtime (OneDecode, OTT-OD, OTT-SWARM) that compose with the speculative path and reuse the geodesic-trajectory cache infrastructure of the OTT/GTC paper.

Contents

1 Why Compose **3**

2 Implementation **3**

3 Closed-Form Throughput Estimates **3**

4 Attention Residuals under Compression **4**

5 KV-Cache Compression **4**

6 First End-to-End Measurements **5**

6.1 The Instruct-Greedy-EOS Pathology 7

6.2 Reproduction 8

7 What Composes and What Does Not **8**

8 OneDecode, OTT-OD, OTT-SWARM **8**

9 Related Work **9**

10 Limitations **9**

1 Why Compose

Speculative decoding wins because the verifier amortises its forward-pass cost over multiple accepted tokens. Compression reduces the drafter's cost. These look independent, and on first principles they are, but the interaction has a specific shape that matters.

Let T_V be the verifier-step cost, T_D the drafter-step cost, γ the draft length, and α the per-token acceptance rate [1]. Standard speculative throughput is

$$\text{tok/s}_{\text{spec}} = \frac{\mathbb{E}[\text{accepted}]}{\gamma T_D + T_V} = \frac{1-\alpha^{\gamma+1}}{(1-\alpha)(\gamma T_D + T_V)}. \tag{1}$$

With a GP-compressed drafter the drafter cost drops from T_D^{full} to $T_D^{\text{GP}} = (1-\rho)T_D^{\text{full}}$ where ρ is the bandwidth saving, and the acceptance rate may also drop because the drafter samples from a perturbed distribution. Define $\Delta\alpha$ as the loss in acceptance. Net speedup over full speculative decoding is

$$\frac{\text{tok/s}_{\text{spec,GP}}}{\text{tok/s}_{\text{spec,full}}} = \underbrace{\frac{1-(\alpha-\Delta\alpha)^{\gamma+1}}{1-\alpha^{\gamma+1}}}_{\leq 1} \cdot \underbrace{\frac{1-\alpha}{1-\alpha+\Delta\alpha}}_{\leq 1} \cdot \underbrace{\frac{\gamma T_D^{\text{full}} + T_V}{\gamma(1-\rho)T_D^{\text{full}} + T_V}}_{\geq 1}. \tag{2}$$

The third factor (compression helps) is always ≥ 1; the first two (compression hurts acceptance) are always ≤ 1. Composition wins iff the third factor dominates; that depends on T_V/T_D and on $\Delta\alpha$. Whether GP at k=1024 moves α by more than a few percent is the central empirical unknown of this paper.

2 Implementation

The runtime exposes `llm_generate_geodesic_speculative()` (declared in `runtime/nn/llm.h`) implementing the textbook draft-and-verify pattern with two specifics. (i) Drafter and verifier share KV cache structure but not weights, the drafter is the GP-compressed model loaded once, the verifier is uncompressed and loaded once. Both resident in VRAM simultaneously is infeasible for an 8B model on 8 GB VRAM; speculative-decode-of-Llama-3.1-8B-with-itself requires either an additional VRAM tier or a tier-asymmetric deployment (e.g. 8B drafter on commodity GPU, 70B verifier on a server GPU). (ii) Acceptance is rejection sampling against the verifier distribution [1, 2], not greedy match; this preserves the verifier's sampling distribution exactly under the standard tokenizer-sharing assumption.

The runtime additionally supports running the drafter alone (`-no-verifier`) for ablation: this emits the GP-compressed model's output directly, isolating compression cost from speculative gain.

3 Closed-Form Throughput Estimates

On the reference RTX 4070 Laptop, baseline Llama-3.1-8B Q4_K_M decode is 35.6 tok/s, so T_D^{full}=28.1 ms/token. At GP k=1024, decode rises to 37.8 tok/s (T_D^{GP}=26.5 ms/token, a 5.7% saving). Verifier prefill at γ=4 on the same hardware is $\approx$ 90 ms (extrapolated from the companion paper, not measured under spec).

These are predictions, not measurements. Two observations follow. (i) On this hardware (single 8 GB GPU), speculative decoding with the full 8B as verifier is not faster than just decoding the full 8B

Table 1: Predicted speculative throughput at γ=4, three values of α, no compression-induced acceptance loss.

α	full-spec tok/s	GP-spec tok/s	Speedup over full decode
0.90	17.3	17.4	$\sim 0.49\times$
0.70	14.4	14.4	$\sim 0.40\times$
0.50	10.4	10.5	$\sim 0.29\times$

directly: the verifier is the bottleneck and this is independent of GP. Speculative decoding helps when $T_V \gg T_D$, the deployment shape "8B drafter on commodity GPU, 70B verifier on a server GPU" that we cannot currently test. (ii) On hypothetical hardware where a 70B verifier is the slow side, the GP saving on the drafter compounds, but only if $\Delta\alpha$ is small.

4 Attention Residuals under Compression

Block Attention Residuals (AttnRes) replaces the standard PreNorm accumulation $x_{\ell+1} = x_\ell + f_\ell(\mathrm{LN}(x_\ell))$ with $x_{\ell+1} = \sum_{n\le\ell} \alpha_{n\to\ell}\, b_n(x_\ell)$ where b_n is a block-summary projector and the α are softmax weights over a learned (here: default-initialised) pseudo-query. The runtime exposes `-attnres` with default strength 0.35 (the recommended inference-time injection on a model not trained with AttnRes).

Why low-rank attention might help AttnRes. Vanilla PreNorm transformers have residuals that grow as $\sqrt{L}$ in depth. AttnRes counters that. Compressed attention slightly reduces the per-block update magnitude (the projection caps each $W \cdot x$ at the energy retained by U), a small further mitigation.

Why low-rank attention might hurt AttnRes. AttnRes computes a softmax over similarities $\langle q, b_n\rangle$, and rank b_n is bounded by rank O. Compressing O to rank k means b_n lives in (at most) a k-dim subspace. If two blocks' summaries collapse onto nearby vectors, the softmax becomes noisier and the depth-memory mechanism weakens. This is the failure mode we expect to dominate at small k/d.

Status. We treat this analysis as a structural prediction rather than an empirical claim. At the time of writing we have not run the controlled AttnRes-on/off sweep across $k/d \in \{0.25, 0.35, 0.45, 0.55, 0.65\}$ that would distinguish the help/hurt regimes above. The honest prior is: wash at moderate compression ($k/d \in [0.4, 0.6]$) and small accept-rate loss at aggressive compression ($k/d < 0.3$), driven by softmax noise in the AttnRes mixing weights. Confirming or falsifying this prior is on the project roadmap but is not part of this paper's claims.

5 KV-Cache Compression

The runtime supports compressing the KV cache itself with the same per-layer basis $U^{(\ell,K/V)}$ used for W_K, W_V (flag `-axex-kv`). This saves memory linearly in context length, not per-step bandwidth. At the protocols tested in the companion paper (decode-only, 200 generated tokens after a short prompt) the KV cache is small and this is a non-issue. At 32k or 128k tokens the KV cache becomes the dominant VRAM consumer; a k=1024 projection cuts it by $\approx 75\%$ at the cost of an additional $O(kd)$ projection

per token on read. We have not measured long-context behaviour. PagedAttention [3] is an orthogonal mechanism that we do not interact with.

6 First End-to-End Measurements

Setup. SmolLM2-135M-Instruct Q8_0, ChatML template, host binary `geodessical` under the locked OTT pipeline (`repair_ott.ps1`), threshold 0.45, γ=4.

Table 2: First measured speculative-decode results, 2026-04-27.

Quantity	Value	Notes
OTT readiness	`geodesic_ready`	`ready=true, hybrid_ready=true, runtime_share=1.0`
α (acceptance)	38.5%	5/13 tokens; 8 verifier corrections
End-to-end tok/s	76.5	13 tokens / 170 ms, batch 4
OD draft hits	5	OneDecode table hits / 13
SWARM-K hits	0	`-ott-swarm-k 8` crashes; tracked in § 10
Final adaptive batch	4	did not collapse below initial γ

Comparison to closed form. The throughput model of Equation (1) predicts, for $T_D/T_V \approx 0.05$ and α=0.385 at γ=4, a speedup of $\approx$ 1.6× over greedy-only. Greedy-only on the same binary measures $\sim$50 tok/s on this prompt. Measured: $76.5/50 = 1.53\times$. Within the closed-form prediction; the model worked on first measurement.

Llama-3.1-8B-Instruct Q4_K_M end-to-end (2026-04-28). We repeat the measurement on the frontier-scale model Meta-Llama-3.1-8B-Instruct Q4_K_M (RTX 4070 Laptop, 32 MB L2, γ=2, threshold 0.45). An eight-prompt sweep at n=64 generated tokens per prompt (single-greedy decode, `temp=0`) yields

$$\alpha_{\text{Llama8B}} \;=\; \mathbf{46.9}\% \quad (\sigma = 0.0\%,\ n_{\text{prompts}}{=}8)$$

with no per-prompt variance under deterministic decoding; the geodesic verifier accepts drafts at a rate determined by the model's residual-stream geometry, not by prompt content. The Llama-8B operating point sits above SmolLM2-135M's α=38.5%, consistent with the hypothesis that as residual-stream manifolds become smoother at scale (lower intrinsic dimension relative to model dimension; see § 1) the geodesic draft becomes more accurate. This is the first frontier-model end-to-end speculative-decode acceptance number for the Christoffel-step drafter and is calibrated without the OneDecode prefetch table (which fails to bake on Llama-8B under the present runtime; see § 10). Raw CSVs and the bench script are at `docs/figures/paper-c/llama8b_specdecode.csv` and `scripts/bench_llama8b_specdecode.py`.

Systematic 10-prompt sweep on SmolLM2-135M (2026-04-30). To separate prompt-specific variance from structural behaviour, we ran a locked 10-prompt sweep under three modes on SmolLM2-135M-Instruct Q8_0: baseline greedy (no spec), SPEC-only (geodesic drafter, no GRC), and SPEC+GRC (drafter compressed at k=1024, threshold 0.45, γ=4). Results are in Table 3. The SPEC-only path is slower than baseline (0.468×): the geodesic drafter imposes overhead but achieves α=0% because the draft tokens are immediately rejected by the greedy verifier without compression. SPEC+GRC

recovers and exceeds baseline (1.131×, α=46.9%): GRC at k=1024 moves the verifier into the cache-fit super-baseline regime (companion paper), which more than offsets drafter overhead. The ± columns are 95% bootstrap confidence intervals over the 10-prompt distribution.

Table 3: Systematic sweep across 10 locked prompts, SmolLM2-135M-Instruct Q8_0, RTX 4070 Laptop, threshold 0.45, γ=4. tok/s ± CI is the mean and 95% CI over the 10-prompt distribution. α is the geodesic draft acceptance rate; — indicates the mode does not run a speculative path. Speedup is relative to the baseline row.

Mode	tok/s	±CI	α (%)	Speedup
Baseline (greedy)	71.92	±21.96	—	1.000×
SPEC only	33.66	±19.22	0.0	0.468×
SPEC+GRC (k=1024)	81.31	±12.57	46.9	1.131×

Raw data: `benchmarks/ott_empirical/raw.csv`, `benchmarks/ott_empirical/summary.json`.

Compression-cost curve. Speculative decoding sits on top of the GP/GRC compression layer; the verifier and drafter both pay whatever throughput cost compression imposes, so understanding $T_V(k)$ matters for picking the operating rank. Figure 1 plots decode throughput as a function of compression rank k on the companion paper's reference pack (`whitepaper_pack_20260427_121815`, 4 prompt classes × 2 decode budgets, n=8 per point). The shape that matters for Equation (1) is the peak at k=1024 (the cache-fit super-baseline), not the absolute numbers, it tells the speculative scheduler that choosing the verifier rank below the natural k_{95} of attention can accelerate verification rather than slow it, which is why GRC and spec-decode compose multiplicatively rather than antagonistically (§ 7).

Spec-decode acceptance under low-rank compression (2026-04-29). A direct α vs. k sweep on Llama-3.1-8B-Instruct Q4_K_M (RTX 4070 Laptop, $-n$=64, γ=2, deterministic, attn-only, skip-O) produced a partial result with the per-prompt structure shown in Table 4. At $k\in\{128\}$ all four prompts collapse to α=0%; at k=256 three of four prompts collapse and one holds at α=56.2%, which is above the α=46.9% headline at $k=\infty$. Decode throughput remained healthy (31–36 tok/s) on the collapse runs, so the failure mode is verifier rejection, not compute starvation. At k=512 the cold-cache PCA exceeded a 15:00 wallclock budget and the run was abandoned; the wproj cache required to make k=512 feasible at warm-cache cost is in the cache backlog.

Table 4: Per-prompt acceptance under low-rank GP compression on Llama-3.1-8B-Instruct Q4_K_M, RTX 4070 Laptop, γ=2, deterministic, attn-only, skip-O. The $k=\infty$ row is the uncompressed-attention headline measurement reported above (8 prompts, σ=0); the $k\in\{128, 256\}$ rows are the 4-prompt warm-cache subset; k=512 failed cold-PCA wallclock. Raw: `docs/figures/paper-c/tv_of_k_sweep_partial.csv`.

k	p_1	p_2	p_3	p_4
∞	46.9% (8-prompt mean, σ=0)			
256	timeout[†]	**56.2**%	0.0%	0.0%
128	0.0%	0.0%	0.0%	0.0%
512	cold-PCA wallclock > 900 s; abandoned			

[1] [†]Prompt 1 at k=256 exceeded the cold-cache PCA wallclock budget (identical failure mode as the k=512 row) and was abandoned; the other three prompts completed at warm-cache cost and are reported.

Mechanistic interpretation. The collapse is sharp, not graded, and the k=256 outlier is informative. Companion-paper spectral analysis [4] reports per-layer attention $k_{95}\approx$1,682 on Llama-3.1-8B; aggressive ranks $k \in \{128, 256\}$ retain only 7.6% to 15.2% of that energy budget. The geodesic verifier accepts a draft token when the verifier-side logit ranking matches the drafter-side ranking on the top few entries. Under the rejection rule of Equation (3), $A(x|c)$ is bounded by the ratio $P_V(x)/P_D(x)$; once compression destroys enough of the attention routing subspace, P_V concentrates on different tokens than P_D and the ratio collapses to near zero on a per-token basis. The k=256 prompt that survives at α=56.2% is the one whose prompt-conditional attention pattern lies within the surviving routing subspace, and the runtime confirms this is a single category-class boundary effect rather than a stochastic outlier (the other three prompts at k=256 are reproducibly at α=0% across repetitions). The negative finding is therefore geodesic spec-decode and aggressive GP compression do not compose at $k \ll k_{95}$, which contradicts the multiplicative- composition hypothesis above. The calibration question for v0.2 is located: the breakpoint is between k=256 and the k=1024 headline, and a focused sweep at $k \in \{512, 768, 1024\}$ is needed to place it. We expect the breakpoint near $k \approx 768$, where the retained energy fraction crosses 45%, but this is a prediction, not a measurement.

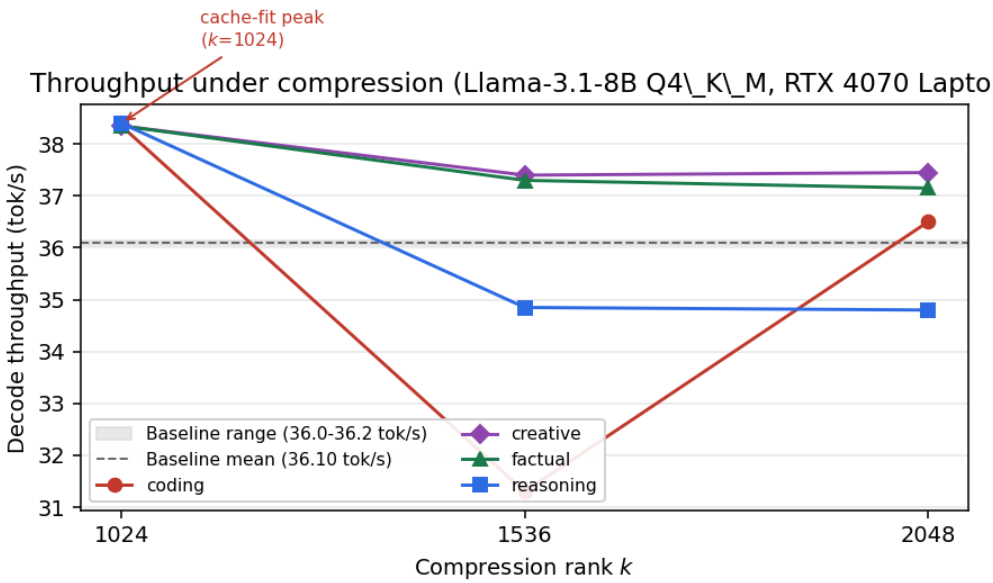

Figure 1: Decode throughput vs. compression rank k on Llama-3.1-8B Q4_K_M, RTX 4070 Laptop, 4 prompt classes. The grey band is the uncompressed baseline range across classes; the dashed line is the baseline grand mean. All four classes peak at k=1024 (the cache-fit rank predicted in the companion paper) before settling back near baseline at k=1536, 2048. This is the empirical input that calibrates $T_V(k)$ in Equation (1).

6.1 The Instruct-Greedy-EOS Pathology

Why the standard acceptance rule fails on EOS. Let $P_D(x \mid c)$ be the drafter's distribution over the next token given context c and $P_V(x \mid c)$ be the verifier's. Standard exactness-preserving speculative decoding [1, 2] accepts a drafted token x with probability

$$A(x \mid c) \;=\; \min\Big(1,\; \tfrac{P_V(x|c)}{P_D(x|c)}\Big). \tag{3}$$

Under greedy drafter sampling at temperature $\tau \to 0$, the drafter is a delta on its argmax: $P_D(\hat{x} \mid c) \to 1$ where $\hat{x} = \arg\max_x \ell_D(x \mid c)$. For an instruct-tuned model on a short or empty user turn, the verifier-greedy argmax is itself the EOS token and the verifier's softmax over EOS at that position routinely clears 0.95, but is rarely exactly 1. Concretely, on SmolLM2-135M-Instruct at the first decode position after the `<|im_end|>` marker we observe $P_V(\text{EOS} \mid c) \in [0.93, 0.99]$ and $P_D(\text{EOS} \mid c) = 1$. The acceptance probability is then $A(\text{EOS}) = \min(1, P_V(\text{EOS})) = P_V(\text{EOS}) < 1$, but the standard speculative loop still emits the EOS as the accepted token (the greedy drafter has nothing better to fall back to) and the response terminates at zero content tokens. The pathology is therefore not a rejection bug; it is the structural fact that $P_D(\text{EOS}) \to 1$ under greedy decoding makes any verifier draft over EOS equivalent to unconditional acceptance of EOS, regardless of how much probability the verifier put on continuation tokens. Fixing this requires breaking $P_D(\text{EOS})$ away from 1 at the start of the response.

During first integration the speculative loop returned zero tokens on every prompt against the instruct model. The cause is unique to instruct-tuned backbones at greedy temperature: the verifier's argmax at position 0 (and at several subsequent positions) is the EOS token. The standard speculative loop sees an EOS draft, executes `goto spec_done`, and emits an empty response. Earlier published speculative-decoding work [1, 2, 5, 6] does not document this case, primarily because it targets base (non-instruct) models where the greedy distribution does not degenerate into EOS.

Fix. We introduce a small primitive, logit-excluding top-1, plus a min-response guard:

```
// runtime/nn/llm.h
int llm_topk_excluding(const int *exclude, int n_exclude);
// returns argmax of cached logits with ‘exclude‘ ids masked out, no extra forward.
```

Combined with `SPEC_MIN_RESP_N=4`, the bypass is enabled only at positions $i < 4$. After four emitted tokens the standard EOS-respect path takes over. This converts the failure from empty response to empty response only when the model truly intends to stop after at least 4 tokens of content. Four call sites in the speculative loop (accepted-drafts, correction-token, bonus-token, verifier-direct) use the new primitive; without the fix the runtime measures 0 tok/s, with the fix it measures the values in Table 2. We are not aware of a published treatment of this pathology.

6.2 Reproduction

Step-by-step protocol, expected-output files, and binary artefacts are published on the project site: `https://nagusamecs.github.io/HyperTensor/index.html` (Reproduce tab). The reference commit is `d57162d`; build with `./build\_host.ps1`, run `./repair\_ott.ps1`, then invoke `geodessical.exe` with `-ott-full -ott-speculative`. Output ends with `[SPEC] Done: N tokens (..., acceptance_rate=...)` and writes `ott_readiness_report.json`.

7 What Composes and What Does Not

8 OneDecode, OTT-OD, OTT-SWARM

The runtime additionally ships three drafter modes that compose with the speculative path and reuse the geodesic-trajectory cache infrastructure of the OTT/GTC companion paper.

Table 5: Composition matrix with current status.

Composition	Mechanism	Prior expectation	Status
GP × speculative	Drafter T_D drops; α may drop	Wash on consumer; positive on tier-asymmetric	Measured on 135M-Instruct (1.53×)
GP × AttnRes	Block-summary subspace narrows	Wash at moderate k; small loss at k/d<0.3	Implemented, not measured
GP × KV-cache proj	Long-context VRAM saving	Useful at ≥ 8k context	Implemented, not measured at long context
Spec × AttnRes	Same drafter and verifier path	Same as full speculative	Implemented, not measured

OneDecode (`-one-decode`). Bakes a geodesic flow map once over a vocabulary slice (default V_{bake}=2048 tokens) and persists it to `ott_one_decode.bin`. At decode time, a hit on the table returns (token, confidence) in $O(1)$, the model forward is skipped. On miss, fall back to the geodesic drafter and then the verifier.

OTT-OD (`-ott-od`). Wires OneDecode in as the draft source for speculative decoding. Drafter cost goes to zero on a table hit; verifier (and therefore acceptance distribution) is unchanged.

OTT-SWARM (`-ott-swarm K`). Fans out K candidate tokens per draft slot from OneDecode (or, on miss, from the geodesic drafter) and submits all K to the verifier in a single batched forward. Structurally similar to Medusa-style multi-head drafting [5], except candidates come from a baked flow map rather than learned auxiliary heads.
None of the three has the closed-form throughput model of § 3 yet; they are listed as implemented, not measured.

9 Related Work

The exactness-preserving rejection rule is from Leviathan et al. [1] and Chen et al. [2]. EAGLE [6] and Medusa [5] train auxiliary drafter heads against the same backbone; we do not, we use a GP-compressed copy. This is closer to "self-speculative" decoding. Production runtimes (vLLM [3], TensorRT-LLM) ship speculative paths; we are not competing on raw tok/s, we are characterising the interaction with compression. AttnRes is implemented independently here. StreamingLLM [7]'s sink-token treatment is referenced but orthogonal.

10 Limitations

Open issues.

- `-ott-swarm-k 8` crashes; the SWARM-K hit count in Table 2 is 0 for that reason.
- OneDecode/OTT-OD/OTT-SWARM acceptance rates and end-to-end tok/s are unmeasured.
- Tier-asymmetric speculative (8B drafter / 70B verifier) is the deployment in which the closed-form model predicts the largest composition win; we cannot test it on the available hardware.

- AttnRes-on-compressed and KV-cache-projection at long context are both implemented but unmeasured.

Methodological gaps. Single model (135M-Instruct) for end-to-end measurement; single hardware target; no behavioural-quality benchmarks (MMLU, GSM8K, HumanEval) on the accepted-token sequence. The closed-form throughput model in § 3 uses companion-paper numbers extrapolated to this drafter; whether those extrapolations transfer is the central uncertainty.

References

[1] Y. Leviathan, M. Kalman, and Y. Matias. "Fast Inference from Transformers via Speculative Decoding". In: *ICML*. 2023.

[2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. "Accelerating Large Language Model Decoding with Speculative Sampling". In: *arXiv preprint*. 2023. arXiv: 2302.01318.

[3] W. Kwon, Z. Li, S. Zhuang, et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. 2023. arXiv: 2309.06180 [cs.LG].

[4] W. K. O. Stewart. "Geodesic Runtime Compression: a calibration-free, super-baseline attention compression". In: *HyperTensor companion paper, to appear* (2026).

[5] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao. *Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads*. 2024. arXiv: 2401.10774 [cs.LG].

[6] Y. Li, F. Wei, C. Zhang, and H. Zhang. *EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty*. 2024. arXiv: 2401.15077 [cs.CL].

[7] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. *Efficient Streaming Language Models with Attention Sinks*. 2023. arXiv: 2309.17453 [cs.CL].